

1. What is Spring Framework?

Spring is a Java-based framework used to build enterprise-level applications. It provides infrastructure support for developing Java applications by offering features like:

- **Dependency Injection (DI)**
- **Aspect-Oriented Programming (AOP)**
- **Transaction Management**
- **MVC Web Framework**
- **Integration with databases (JDBC, JPA, Hibernate)**
- **Security (Spring Security)**

Spring helps you develop loosely coupled, easily testable, and maintainable applications.

2. What is Spring Boot?

Spring Boot is a project built on top of Spring to simplify Spring application development. It does this by:

- **Reducing boilerplate code**
- **Providing opinionated defaults and auto-configuration**
- **Embedding servers like Tomcat or Jetty**
- **Offering production-ready features like metrics, health checks, etc.**

Spring Boot makes it easy to create standalone, production-ready Spring applications with minimal configuration.

3. Spring Boot Internal Working (Simplified)

When you run a Spring Boot application:

Step-by-step internal working:

1. **@SpringBootApplication annotation triggers:**

- **@ComponentScan:** Scans for components, services, controllers.
 - **@EnableAutoConfiguration:** Triggers Spring Boot's auto-configuration logic.
 - **@Configuration:** Marks the class as a source of bean definitions.
2. **AutoConfiguration** loads based on classpath contents using:
 - **spring.factories** file in META-INF
 - **Example:** If **spring-boot-starter-data-jpa** is on the classpath, Spring Boot configures JPA repositories automatically.
 3. **Embedded server** (like Tomcat) is started internally.
 4. **ApplicationContext** is initialized (IoC container).
 5. All beans are created and injected.
 6. Application starts serving HTTP requests (if web app).
-

4. What is IoC Container?

IoC (Inversion of Control) Container is the core of the Spring Framework. It manages:

- **Object creation**
- **Object lifecycle**
- **Dependency injection** Spring's IoC container is implemented by **ApplicationContext**.

It creates and manages objects (beans), resolves dependencies, and wires them together.

5. What is Dependency Injection (DI)?

Dependency Injection is a design pattern where dependencies are injected by the container rather than the object creating them manually.

Example:

```
class Car {  
  
    private Engine engine;
```

// Dependency injected via constructor

```
public Car(Engine engine) {  
    this.engine = engine;  
}  
}
```

In Spring:

@Component

class Engine {}

@Component

```
class Car {  
    private final Engine engine;  
    @Autowired  
    public Car(Engine engine) {  
        this.engine = engine;  
    }  
}
```

Spring injects Engine into Car without you writing new Engine().

Types of DI in Spring:

- **Constructor Injection (recommended)**
- **Setter Injection**
- **Field Injection (not preferred)**

3. DispatcherServlet & web.xml – Internal Working DispatcherServlet – What is it?

It's the Front Controller in Spring MVC. All HTTP requests pass through it.

Internal Request Flow:

[Client Request]

|

v

[DispatcherServlet]

|

v

[HandlerMapping] --> Finds correct controller

|

v

[Controller]

|

v

[Service / DAO]

|

v

[Return ModelAndView or JSON]

|

v

[ViewResolver]

|

v

[HTML/JSON Response]

What is web.xml?

In classic Java web apps, web.xml defined servlet mappings:

xml

Copy

Edit

<servlet>

<servlet-name>spring</servlet-name>

```
<servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet-class>

</servlet>

<servlet-mapping>

<servlet-name>spring</servlet-name>

<url-pattern>/</url-pattern>

</servlet-mapping>
```

1. Spring Profiles & Environment Management

What is a Spring Profile?

A Profile in Spring is a way to segregate configuration for different environments (e.g., dev, test, prod). You can define beans, properties, or configurations that should only be active in a specific environment.

Example:

@Configuration

@Profile("dev")

public class DevConfig {

@Bean

public DataSource dataSource() {

// return H2 database for dev

}

}

@Configuration

@Profile("prod")

public class ProdConfig {

@Bean

public DataSource dataSource() {

// return real DB for prod

}

```
}
```

How to activate a profile:

- `application.properties / yml`

`spring.profiles.active=dev`

- **Command line**

`java -jar myapp.jar --spring.profiles.active=prod`

- **Programmatically**

`SpringApplication app = new SpringApplication(MyApp.class);`

`app.setAdditionalProfiles("test");`

`app.run(args);`

Environment Management

Spring Boot uses the Environment abstraction to manage:

- **Active profiles**
- **Property sources (application.properties/yml, system properties, env vars, etc.)**

Accessing environment:

@Autowired

Environment env;

```
public void printDbUrl() {
```

```
    System.out.println(env.getProperty("spring.datasource.url"));
```

```
}
```

2. CommandLineRunner vs ApplicationRunner

These are callback interfaces used to run code after the Spring Boot application starts, often used for:

- Preloading data
- Logging
- Running setup tasks

CommandLineRunner

- Runs after the ApplicationContext is loaded
- Accepts raw command-line arguments

@Component

```
public class MyCommandLineRunner implements CommandLineRunner {

    @Override

    public void run(String... args) {

        System.out.println("CommandLineRunner executed!");

        for (String arg : args) {

            System.out.println("Arg: " + arg);

        }

    }

}
```

ApplicationRunner

- Similar to CommandLineRunner but provides access to parsed arguments
- Uses ApplicationArguments which gives options and non-option args

@Component

```
public class MyApplicationRunner implements ApplicationRunner {

    @Override

    public void run(ApplicationArguments args) {

        System.out.println("ApplicationRunner executed!");

        System.out.println("Non-option args: " + args.getNonOptionArgs());

        System.out.println("Option names: " + args.getOptionNames());

    }

}
```

```
}  
}
```

1. Spring Profiles & Environment Management

What is a Spring Profile?

A Profile in Spring is a way to segregate configuration for different environments (e.g., dev, test, prod). You can define beans, properties, or configurations that should only be active in a specific environment.

Example:

```
@Configuration
```

```
@Profile("dev")
```

```
public class DevConfig {
```

```
    @Bean
```

```
    public DataSource dataSource() {
```

```
        // return H2 database for dev
```

```
    }
```

```
}
```

```
@Configuration
```

```
@Profile("prod")
```

```
public class ProdConfig {
```

```
    @Bean
```

```
    public DataSource dataSource() {
```

```
        // return real DB for prod
```

```
    }
```

```
}
```

How to activate a profile:

- `application.properties / yml`

`spring.profiles.active=dev`

- **Command line**

`java -jar myapp.jar --spring.profiles.active=prod`

- **Programmatically**

```
SpringApplication app = new SpringApplication(MyApp.class);
```

```
app.setAdditionalProfiles("test");
```

```
app.run(args);
```

Environment Management

Spring Boot uses the Environment abstraction to manage:

- **Active profiles**
- **Property sources** (`application.properties/yml`, system properties, env vars, etc.)

Accessing environment:

@Autowired

Environment env;

```
public void printDbUrl() {
```

```
    System.out.println(env.getProperty("spring.datasource.url"));
```

```
}
```

2. CommandLineRunner vs ApplicationRunner

These are callback interfaces used to run code after the Spring Boot application starts, often used for:

- **Preloading data**
- **Logging**
- **Running setup tasks**

CommandLineRunner

- Runs after the ApplicationContext is loaded
- Accepts raw command-line arguments

@Component

```
public class MyCommandLineRunner implements CommandLineRunner {
```

@Override

```
public void run(String... args) {
```

```
    System.out.println("CommandLineRunner executed!");
```

```
    for (String arg : args) {
```

```
        System.out.println("Arg: " + arg);
```

```
    }
```

```
}
```

```
}
```

ApplicationRunner

- Similar to CommandLineRunner but provides access to parsed arguments
- Uses ApplicationArguments which gives options and non-option args

@Component

```
public class MyApplicationRunner implements ApplicationRunner {
```

@Override

```
public void run(ApplicationArguments args) {
```

```
    System.out.println("ApplicationRunner executed!");
```

```
    System.out.println("Non-option args: " + args.getNonOptionArgs());
```

```
    System.out.println("Option names: " + args.getOptionNames());
```

```
}
```

}

1. What is a @Controller?

- A class annotated with @Controller is a web controller in Spring MVC.
- It handles HTTP requests, processes data (with services), and returns a view (HTML/JSP).

@Controller

```
public class WebController {  
  
    @GetMapping("/hello")  
  
    public String hello(Model model) {  
        model.addAttribute("name", "Spring");  
        return "hello"; // returns hello.html or hello.jsp  
    }  
}
```

Use this when building web pages with templates (Thymeleaf, JSP, etc.).

2. What is a @RestController?

- Shortcut for @Controller + @ResponseBody
- It tells Spring to return JSON/XML data directly instead of a view.
- Ideal for REST APIs.

@RestController

```
public class ApiController {  
  
    @GetMapping("/api/greet")  
  
    public String greet() {  
        return "Hello from API";  
    }  
}
```

Use this when building RESTful services.

3. Handling Different HTTP Methods

GET – Read/Fetch

@GetMapping("/users")

public List<User> getAllUsers() { ... }

POST – Create

@PostMapping("/users")

public User createUser(@RequestBody User user) { ... }

PUT – Update

@PutMapping("/users/{id}")

public User updateUser(@PathVariable Long id, @RequestBody User user) { ... }

DELETE – Remove

@DeleteMapping("/users/{id}")

public void deleteUser(@PathVariable Long id) { ... }

4. Creating and Using ResponseEntity

ResponseEntity<T> allows you to:

- Control HTTP status code
- Add headers
- Return body

Example with status and body

@GetMapping("/users/{id}")

public ResponseEntity<User> getUser(@PathVariable Long id) {

User user = userService.findById(id);

if (user != null) {

return ResponseEntity.ok(user); // 200 OK

} else {

```
return ResponseEntity.status(HttpStatus.NOT_FOUND).build(); // 404
}
}
```

Example with headers

```
@GetMapping("/hello")
public ResponseEntity<String> hello() {
    HttpHeaders headers = new HttpHeaders();
    headers.add("Custom-Header", "value");
    return new ResponseEntity<>("Hello World", headers, HttpStatus.OK);
}
```

REST API Validation in Spring Boot REST API validation ensures the data received from the client is correct before processing it. Spring Boot makes it simple using Java Bean Validation (JSR-380) with annotations like `@NotNull`, `@Size`, etc., and the use of `@Valid`.

1. Enable Validation Dependencies

Add to pom.xml (if you're using Spring Boot Starter Web, this is already included):

```
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-validation</artifactId>
</dependency>
```

2. Apply Validation Annotations

Create a DTO (Data Transfer Object) with constraints:

```
public class UserDTO {
    @NotNull(message = "Name cannot be null")
    @Size(min = 2, message = "Name must be at least 2 characters")
    private String name;
```

```
@Email(message = "Invalid email format")

private String email;

@Min(value = 18, message = "Age must be at least 18")

private int age;

// Getters and setters

}
```

3. Validate in Controller Using @Valid

```
@RestController

@RequestMapping("/users")

public class UserController {

    @PostMapping

    public ResponseEntity<String> createUser(@Valid @RequestBody UserDTO user) {

        // if valid, process user

        return ResponseEntity.ok("User created");

    }

}
```

4. Handle Validation Errors with @ControllerAdvice

Spring will throw `MethodArgumentNotValidException` for validation errors. Handle it globally:

```
@ControllerAdvice

public class GlobalExceptionHandler {

    @ExceptionHandler(MethodArgumentNotValidException.class)

    public ResponseEntity<Map<String, String>>

    handleValidationErrors(MethodArgumentNotValidException ex) {
```

```
Map<String, String> errors = new HashMap<>();  
ex.getBindingResult().getFieldErrors().forEach(error -> {  
    errors.put(error.getField(), error.getDefaultMessage());  
});  
return ResponseEntity.badRequest().body(errors);  
}  
}
```

Sample JSON Request & Response

Valid Request:

```
{  
  "name": "Alice",  
  "email": "alice@example.com",  
  "age": 22  
}
```

Invalid Response:

```
{  
  "name": "Name must be at least 2 characters",  
  "email": "Invalid email format",  
  "age": "Age must be at least 18"  
}
```

What is Spring AOP?

Spring AOP is a programming paradigm that allows you to modularize concerns that cut across multiple classes (like logging, security, or transactions). Instead of repeating this logic in every class, you can write it once in a reusable module called an "Aspect."

Key Concepts in AOP:

Term	Meaning
Aspect	A module/class that encapsulates cross-cutting logic (e.g., logging)
Advice	The actual action taken (method) at a specific point (e.g., before method)
JoinPoint	A point in program execution (e.g., method call) where advice can run
Pointcut	An expression that selects join points where advice should apply
Proxy	A wrapper object created by Spring to apply AOP logic around beans

1. Aspect – The cross-cutting logic container

@Aspect

@Component

```
public class LoggingAspect {
    @Before("execution(* com.example.service.*(..))")
    public void logBefore(JoinPoint joinPoint) {
        System.out.println("Logging before: " + joinPoint.getSignature().getName());
    }
}
```

Here, `LoggingAspect` is an Aspect — it wraps logic around service methods.

2. Advice – What to do and when

Advice is what kind of logic to run and when. Spring supports:

Advice Type	When it runs
@Before	Before method execution

Advice Type	When it runs
@After	After method finishes (both success/fail)
@AfterReturning	After method returns successfully
@AfterThrowing	When method throws an exception
@Around	Wraps method — before & after (most powerful)

3. JoinPoint – Where the advice is applied

- A join point is a point during execution like a method call or exception throw.
- Spring AOP only supports method execution join points (because it's proxy-based). Example usage:

```
public void logMethodCall(JoinPoint joinPoint) {
    System.out.println("Called method: " + joinPoint.getSignature().getName());
}
```

4. Pointcut – Defines *where* to apply advice

You define pointcut expressions to tell Spring *which methods* should be intercepted.

```
@Before("execution(* com.example.service.*(..))")
public void beforeAdvice() {
    // This will run before any method in the service package
}
```

Common Pointcut Expressions

Expression	Meaning
execution(* pkg.Class.method(..))	Specific method
execution(* pkg.*(..))	All methods in package

Expression	Meaning
within(pkg.Class)	Any method inside the class
this(bean) / target(bean)	Bean or its proxy object
@annotation(AnnotationName)	Methods annotated with given annotation
args(type)	Methods taking specific argument types

5. How Spring AOP Works Internally – Proxies

Spring AOP works using dynamic proxies:

Proxy Type	When Used
JDK Proxy	If the target class implements an interface
CGLIB Proxy	If the target class does not implement interface (uses subclassing)

- Proxy wraps your bean and intercepts method calls to inject AOP logic.
- Example: You call `userService.getUser()` → Proxy intercepts and runs `@Before`, `@After`, etc.

Diagram (Simplified Flow)

Your Code -----> Proxy -----> Real Bean

↑ ↓

AOP Advice Before/After

Example Use Case

@Aspect

@Component

```
public class SecurityAspect {
```

```
    @Before("execution(* com.example.service.*(..))")
```

```
public void checkAuthentication() {  
    System.out.println("Security check before method");  
}  
}
```

Whenever any method in `com.example.service` is called, Spring:

1. Passes the call to the proxy
2. Executes the `checkAuthentication()` advice

What is Spring Data JPA?

Spring Data JPA is a part of the **Spring Data** project that makes it easier to implement **JPA-based data access layers** in Spring applications. It provides a high-level abstraction over the Java Persistence API (JPA), allowing developers to interact with databases with minimal boilerplate code.

◆ 1. `CrudRepository<T, ID>`

Purpose: Basic CRUD operations.

Generic Parameters:

- T – Entity type
- ID – Type of the entity's primary key

Key Methods:

```
<S extends T> S save(S entity);
```

```
Optional<T> findById(ID id);
```

```
boolean existsById(ID id);
```

```
Iterable<T> findAll();
```

```
void deleteById(ID id);
```

```
void delete(T entity);
```

Usage Example:

```
public interface UserRepository extends CrudRepository<User, Long> {
```

```
}
```

✅ Use when you just need simple CRUD functionality.

◆ 2. PagingAndSortingRepository<T, ID>

Extends: CrudRepository

Purpose: Adds support for **pagination** and **sorting**.

Key Methods:

```
Iterable<T> findAll(Sort sort);
```

```
Page<T> findAll(Pageable pageable);
```

Usage Example:

```
public interface UserRepository extends PagingAndSortingRepository<User, Long> {  
}
```

Pagination Example:

```
Pageable pageable = PageRequest.of(0, 10, Sort.by("name").ascending());
```

```
Page<User> users = userRepository.findAll(pageable);
```

✅ Use when you need **pagination and sorting** on top of basic CRUD.

◆ 3. JpaRepository<T, ID>

Extends: PagingAndSortingRepository

Purpose: Full feature-rich JPA repository. Adds more **JPA-specific** functionality.

Extra Methods:

```
List<T> findAll();
```

```
List<T> findAllById(Iterable<ID> ids);
```

```
void deleteInBatch(Iterable<T> entities);
```

```
<S extends T> List<S> saveAll(Iterable<S> entities);
```

Usage Example:

```
public interface UserRepository extends JpaRepository<User, Long> {  
}
```

✓ Use when you want **all JPA features**, including batch operations, flush(), custom queries, etc.

Interface Hierarchy Diagram

JpaRepository

↑

PagingAndSortingRepository

↑

CrudRepository

Spring Data JPA – Database Configuration (MySQL)

Basic application.properties Example:

```
spring.datasource.url=jdbc:mysql://localhost:3306/schooldb
```

```
spring.datasource.username=root
```

```
spring.datasource.password=admin
```

```
spring.datasource.driver-class-name=com.mysql.jdbc.Driver
```

```
spring.jpa.hibernate.ddl-auto=update
```

```
spring.jpa.show-sql=true
```

Property Breakdown

✓ **spring.jpa.hibernate.ddl-auto=update**

- Automatically updates the DB schema based on entity changes.
- Does **not** delete existing data.
- Useful for **development**, avoid in **production**.

Option	Behavior
none	No schema action
validate	Validates schema, throws if mismatch
update	Updates schema non-destructively
create	Drops and re-creates schema at start
create-drop	Same as create but drops on shutdown

spring.datasource.url

- JDBC URL to connect to the MySQL database.
- Example: jdbc:mysql://localhost:3306/schooldb

spring.datasource.username / spring.datasource.password

- Database login credentials.

spring.datasource.driver-class-name

- JDBC driver class for MySQL.
- Old: com.mysql.jdbc.Driver (use only with MySQL Connector 5.x)
- Recommended (for 6.x+): com.mysql.cj.jdbc.Driver

spring.jpa.show-sql=true

- Logs all SQL queries generated by Hibernate.
- Helps in debugging and SQL verification.

Best Practices

- Prefer spring.jpa.hibernate.ddl-auto=validate or none in **production**.
- Use environment variables or secrets for DB credentials.

- Format SQL logs with:
- `spring.jpa.properties.hibernate.format_sql=true`

Spring Data JPA Annotations Explained

◆ @Entity

Marks a Java class as a **JPA entity** (i.e., a table in the database).

@Entity

```
public class Course {  
    // Fields mapped to columns  
}
```

- It **must have a no-arg constructor**.
- The class is mapped to a table (default name = class name in lowercase unless overridden by @Table).

◆ @Table(name = "table_name")

Specifies the **database table name** to map the entity.

@Table(name = "courses")

If omitted, JPA uses the class name (Course → course) as the table name.

◆ @Column(name = "column_name")

Specifies the mapping between a field and a **column** in the database.

@Column(name = "course_name", nullable = false, unique = true)

private String name;

You can customize column properties like:

- nullable

- length
- unique
- columnDefinition (for custom SQL types)

◆ @Id

Marks a field as the **primary key** of the entity.

@Id

private Long id;

Must be used with @GeneratedValue if you want auto-generated IDs.

◆ @GeneratedValue – Auto-Generating Primary Keys

Used to specify **how the primary key should be generated**.

@GeneratedValue(strategy = GenerationType.SEQUENCE, generator = "course_sequence")

🌀 Strategy Options:

Strategy	Description
AUTO	JPA picks best strategy based on DB
IDENTITY	Uses DB's auto-increment feature
SEQUENCE	Uses a database sequence
TABLE	Uses a separate table to maintain ID values (not recommended for large scale)

◆ @SequenceGenerator

Defines a **sequence** in the database for use with GenerationType.SEQUENCE.

@SequenceGenerator(

```
name = "course_sequence",      // reference name
sequenceName = "course_sequence", // actual DB sequence name
allocationSize = 1             // increment by 1
)
```

✅ How to Use an Existing Sequence

If the sequence is already created in the database, you can just reference it:

SQL

```
CREATE SEQUENCE course_sequence START WITH 1 INCREMENT BY 1;
```

✅ Entity Code Example

```
@Entity
```

```
@Table(name = "courses")
```

```
public class Course {
```

```
    @Id
```

```
    @SequenceGenerator(
```

```
        name = "course_sequence",
```

```
        sequenceName = "course_sequence",
```

```
        allocationSize = 1
```

```
    )
```

```
    @GeneratedValue(
```

```
        strategy = GenerationType.SEQUENCE,
```

```
        generator = "course_sequence"
```

```
    )
```

```
    private Long id;
```

```
@Column(name = "course_name", nullable = false)
private String name;

// Constructors, Getters, Setters
}
```

◆ @Repository

Marks a class as a **DAO** (Data Access Object) and enables automatic exception translation.

```
@Repository
public interface CourseRepository extends JpaRepository<Course, Long> {
}
```

- Usually not required explicitly when extending JpaRepository — Spring handles it.
-

◆ @Service

Marks a class as a **service layer bean**, typically containing business logic.

```
@Service
public class CourseService {
    @Autowired
    private CourseRepository courseRepository;
}
```

Spring Data JPA – Custom Queries & Annotations

✓ @Query – Custom Queries

1. JPQL Query

```
@Query("select s.firstName from Student s where s.emailId = ?1")
```

String getStudentFirstNameByEmailAddress(String emailId);

- JPQL is entity-based, not table-based.
- ?1, ?2, etc., denote positional parameters.
- Used to retrieve specific fields or complex query logic.

2. Native SQL Query

@Query(value = "SELECT * FROM tbl_student s where s.email_address = ?1", nativeQuery = true)

Student getStudentByEmailAddressNative(String emailId);

- Uses actual DB table and column names.
- nativeQuery = true is required for raw SQL.

3. Native Query with Named Parameters

@Query(value = "SELECT * FROM tbl_student s where s.email_address = :emailId", nativeQuery = true)

Student getStudentByEmailAddressNativeNamedParam(@Param("emailId") String emailId);

- Uses named parameters (:paramName) for better readability.
- Must use @Param("name") to bind parameters.

@Modifying + @Transactional – For Updates

@Modifying

@Transactional

@Query(value = "update tbl_student set first_name = ?1 where email_address = ?2", nativeQuery = true)

int updateStudentNameByEmailId(String firstName, String emailId);

- @Modifying: Required for INSERT, UPDATE, DELETE.
- @Transactional: Ensures DB transaction is handled.
- Returns number of rows affected.

Spring Data JPA – Repository Methods (Default + Custom)

✓ 1. Default Methods from JpaRepository

By extending JpaRepository<Entity, ID>, you get these built-in methods:

```
List<T> findAll();
```

```
T getById(ID id);
```

```
Optional<T> findById(ID id);
```

```
T save(T entity);
```

```
void deleteById(ID id);
```

```
boolean existsById(ID id);
```

```
long count();
```

Also includes:

```
List<T> findAll(Sort sort);
```

```
Page<T> findAll(Pageable pageable);
```

✓ 2. Custom Finder Methods (Derived Query Methods)

You can define methods by simply **following naming conventions**, and Spring will generate the queries automatically.

Naming Syntax:

```
findBy<FieldName><Condition>
```

Examples:

```
List<Student> findByFirstName(String firstName);
```

```
Student findByEmailId(String emailId);
```

```
List<Student> findByFirstNameAndLastName(String firstName, String lastName);
```

```
List<Student> findByFirstNameContaining(String keyword);
```

```
List<Student> findByAgeGreaterThan(int age);
```

Keywords Supported

Keyword	Example Method
And / Or	findByFirstNameAndLastName
Is, Equals	findByEmailIdEquals
Between	findByAgeBetween(int min, int max)
LessThan / GreaterThan	findByAgeGreaterThan(20)
Like / NotLike	findByNameLike("%abc%")
In / NotIn	findByIdIn(List<Long> ids)
OrderBy	findByFirstNameOrderByAgeDesc()
IsNull / IsNotNull	findByLastNameIsNull()
Top, First	findTop3ByAgeGreaterThan(25)
Distinct	findDistinctByLastName(String name)

Spring Data JPA – @Embedded, @Embeddable, and @AttributeOverride(s)

1. @Embeddable

- Marks a **value type class** that can be embedded into an entity.

- Does **not have its own table**; fields are stored in the parent table.

@Embeddable

```
public class Guardian {  
    private String name;  
    private String email;  
    private String mobile;  
}
```

✓ 2. @Embedded

- Used in an @Entity to include the fields from an @Embeddable class.
- All fields of the embedded class are **flattened into the entity table**.

@Embedded

```
private Guardian guardian;
```

✓ 3. @AttributeOverride / @AttributeOverrides

- Used to **override column names or properties** of embedded fields.
- Helpful when:
 - Embedding the same object more than once
 - DB column names differ from field names

@Embedded

```
@AttributeOverrides({  
    @AttributeOverride(name = "name", column = @Column(name = "guardian_name")),  
    @AttributeOverride(name = "email", column = @Column(name = "guardian_email")),  
    @AttributeOverride(name = "mobile", column = @Column(name = "guardian_mobile"))  
})  
  
private Guardian guardian;
```

✓ Practical Example

Guardian.java

@Embeddable

```
public class Guardian {  
    private String name;  
    private String email;  
    private String mobile;  
}
```

Student.java

@Entity

```
public class Student {
```

@Id

@GeneratedValue(strategy = GenerationType.IDENTITY)

private Long id;

private String firstName;

@Embedded

@AttributeOverrides({

@AttributeOverride(name = "name", column = @Column(name = "guardian_name")),

@AttributeOverride(name = "email", column = @Column(name = "guardian_email")),

@AttributeOverride(name = "mobile", column = @Column(name = "guardian_mobile"))

})

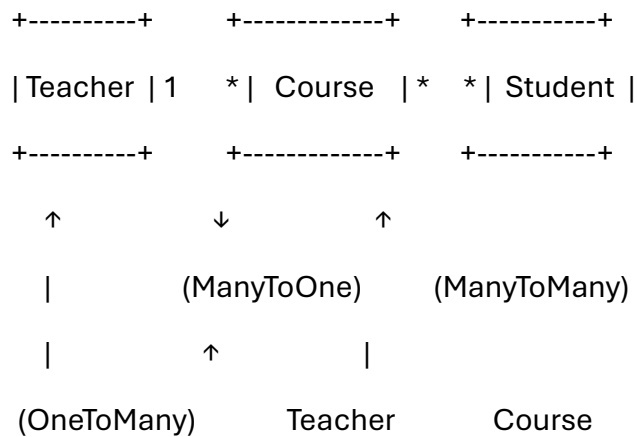
private Guardian guardian;

}

Key Points

Annotation	Purpose
@Embeddable	Marks a class as embeddable (reusable value type)
@Embedded	Embeds an @Embeddable into an entity
@AttributeOverride	Overrides default column mapping for embedded field
@AttributeOverrides	Allows multiple overrides

ER Diagram – Relationships



Relationship Summary:

- **Teacher → Course:** @OneToMany / @ManyToOne
- **Student ↔ Course:** @ManyToMany with join table

Entity Relationship Explanation

@ManyToOne

@ManyToOne(cascade = CascadeType.ALL)

```
@JoinColumn(name = "teacher_id", referencedColumnName = "teacherId")
```

```
private Teacher teacher;
```

- **Relationship:** Many Courses are taught by one Teacher.
- @JoinColumn defines the **foreign key** (teacher_id) in course table.
- Belongs on the **"many" side** (i.e., Course).
- cascade = CascadeType.ALL: any operation (persist/delete/etc.) on Course cascades to Teacher.

✚ **When to Use:** You want to link many entities to one parent (e.g., multiple courses under one teacher).

✅ @OneToMany

```
@OneToMany(mappedBy = "teacher", cascade = CascadeType.ALL)
```

```
private List<Course> courses;
```

- Placed in Teacher entity.
- mappedBy = "teacher" points to the field in Course.
- Inverse side of @ManyToOne.

✚ **When to Use:** You want to navigate from Teacher to all their Courses.

✅ @ManyToMany

```
@ManyToMany(cascade = CascadeType.ALL)
```

```
@JoinTable(
```

```
    name = "studen_course_map",
```

```
    joinColumns = @JoinColumn(name = "course_id", referencedColumnName =  
"courseId"),
```

```
    inverseJoinColumns = @JoinColumn(name = "student_id", referencedColumnName =  
"studentId")
```

```
)
```

```
private List<Student> students;
```

- @ManyToMany needs a **join table**.
- You define @JoinTable in **one of the entities**, typically the **owning side**.
- joinColumns → this class's foreign key (course_id)
- inverseJoinColumns → related class's foreign key (student_id)

📌 **When to Use:** You want to map many-to-many (e.g., many students can take many courses).

🧠 Where to Put @ManyToMany?

- Either side can own the relationship, but typically you put @ManyToMany + @JoinTable in the entity that **logically owns** the relationship.
 - If both sides are equal, use the side where you'll **manage (add/remove)** the relationship more often.
-

✂ Sample Entities Overview

◆ Teacher.java

@Entity

```
public class Teacher {
```

```
    @Id
```

```
    private Long teacherId;
```

```
    @OneToMany(mappedBy = "teacher", cascade = CascadeType.ALL)
```

```
    private List<Course> courses;
```

```
}
```

◆ Course.java

@Entity

```

public class Course {

    @Id

    private Long courseId;


    @ManyToOne(cascade = CascadeType.ALL)

    @JoinColumn(name = "teacher_id", referencedColumnName = "teacherId")

    private Teacher teacher;


    @ManyToMany(cascade = CascadeType.ALL)

    @JoinTable(

        name = "studen_course_map",

        joinColumns = @JoinColumn(name = "course_id", referencedColumnName =
"courseId"),

        inverseJoinColumns = @JoinColumn(name = "student_id", referencedColumnName =
"studentId")

    )

    private List<Student> students;
}

```

◆ Student.java

```

@Entity

public class Student {

    @Id

    private Long studentId;


    @ManyToMany(mappedBy = "students") // inverse side

    private List<Course> courses;
}

```

```
}
```

What is cascade in JPA?

In JPA, **cascade operations** allow you to **automatically propagate** certain operations (like save, delete, merge) from a **parent entity to its child/related entities**.

This helps maintain **data integrity** and reduce boilerplate code when dealing with **entity relationships** like @OneToMany, @ManyToOne, etc.

Cascade Types in JPA

✅ 1. CascadeType.PERSIST

Behavior: When you call persist() or save() on the parent entity, **all child entities are also persisted** (inserted into the DB).

Use Case: You're creating a new parent and new child entities **at the same time**.

💡 Example: You create a new Order with a list of OrderItems. Saving the Order will also insert all OrderItems.

✅ 2. CascadeType.MERGE

Behavior: When the parent entity is updated (merged), the **changes in child entities are also merged** into the DB.

Use Case: Used when you're updating detached entities that include nested objects.

💡 Example: You load a User, detach it from the session, change their Address, then merge the User again. This also updates the Address entity.

✅ 3. CascadeType.REMOVE

Behavior: When the parent entity is deleted, all related child entities are **automatically deleted**.

Use Case: Used when child entities shouldn't exist without their parent.

⚠ Be cautious! It can lead to **unintentional data loss**.

💡 Example: Deleting a Department should also delete all Employees under it — only if they aren't shared with any other entity.

✅ 4. CascadeType.REFRESH

Behavior: When you refresh the parent entity (reload from DB), the **child entities are also refreshed**.

Use Case: Ensures both parent and related entities have the **most up-to-date** data from the database.

💡 Example: You refresh an Invoice to get the latest changes — its associated PaymentDetails are also reloaded.

✅ 5. CascadeType.DETACH

Behavior: When the parent is detached from the persistence context, the **child entities are also detached**.

Use Case: Used in advanced scenarios involving the entity lifecycle (e.g., manually detaching objects from Hibernate session).

💡 Example: You manually detach a Project from the entity manager — the associated Tasks are also detached.

✅ 6. CascadeType.ALL

Behavior: Applies **all** of the above cascade types: PERSIST, MERGE, REMOVE, REFRESH, DETACH.

Use Case: Used when the child entity's lifecycle is **completely dependent** on the parent.

💡 Example: A Book and its Pages. Pages have no meaning outside the book. You want all actions on the book to cascade to pages.

⚠ Important Tips

- **Do not blindly use CascadeType.ALL** — especially with REMOVE. Always think about how child entities should behave independently.
- In **ManyToOne** relationships (e.g., Student -> Course), avoid cascading REMOVE unless you're absolutely sure.
- Cascading is most commonly used in:
 - @OneToMany
 - @OneToOne
 - (less often) @ManyToMany

What is Transaction Management in JPA?

Transaction Management ensures that a **set of operations either fully complete or fully fail** — preserving **data consistency** and **integrity**.

In JPA (via Hibernate), a transaction wraps around **database operations** such as insert, update, delete, etc.

Internal Working of Transactions in JPA

Step-by-Step Execution:

1. Transaction Start

- When a method annotated with @Transactional is called:
 - Spring AOP creates a proxy and begins a transaction via EntityManager.getTransaction().begin() (for resource-local) or through JTA.
 - A **Hibernate session** is also started.

2. Persistence Context is Bound

- JPA binds the EntityManager to the current transaction.
- All entities manipulated are **tracked** in the persistence context (a 1st-level cache).

3. DB Operations are Deferred

- Changes (persist, merge, remove) are **queued**.

- SQL is not immediately sent — instead, it's flushed **before commit**.

4. Transaction Commit

- At the end of method execution (if no exception):
 - JPA flushes changes to the DB.
 - Hibernate executes SQL statements.
 - `EntityManager.getTransaction().commit()` is called.

5. Rollback on Exception

- If an exception (usually unchecked) is thrown:
 - `EntityManager.getTransaction().rollback()` is called.
 - Changes in memory (persistence context) are discarded.
 - DB remains unchanged.

How Spring Simplifies It

@Transactional

@Transactional

```
public void createStudent(Student student) {  
    studentRepository.save(student);  
}
```

- Spring uses AOP to wrap the method in a transactional context.
- Default rollback happens only on **unchecked exceptions** (`RuntimeException` or `Error`).

What is Transaction Propagation?

In Spring, **transaction propagation** defines **how a method should behave if a transaction already exists** when it is called.

It answers:

"Should this method **join** the existing transaction, **start** a new one, or **run without** any transaction?"

✓ All Transaction Propagation Types (with Examples)

Propagation Type	Description
REQUIRED	Join current, or create new if none exists
REQUIRES_NEW	Always suspend current and start a new one
NESTED	Start a nested transaction within current
MANDATORY	Must run within existing transaction
NEVER	Must run without a transaction
SUPPORTS	Run inside transaction if present , else run non-transactionally
NOT_SUPPORTED	Suspend existing transaction and run non-transactionally

Thank you — let's simplify this by building a **clear, step-by-step example** using real-world scenarios so you **fully understand what happens in parent vs child transactions** and how **transaction propagation** works practically.

🧠 Setup for Understanding Propagation (Real-World Inspired)

Let's assume:

- You are building a **Banking Application**.
- You have a **BankService** class that transfers money.
- Inside it, you call two methods:
 - `debitAccount()` – subtracts money from the sender
 - `creditAccount()` – adds money to the receiver

We'll treat transferMoney() as the **parent method**,
and debitAccount() / creditAccount() as **child methods**.

✅ **Let's build on this method:**

```
public void transferMoney() {  
    debitAccount();  
    creditAccount();  
}
```

Now we'll see what happens based on different propagation types applied to debitAccount() and creditAccount().

🔄 **Propagation Scenarios (With Explanation)**

✅ **1. REQUIRED (Default)**

Child joins parent's transaction.

```
@Transactional // Parent  
  
public void transferMoney() {  
    debitAccount(); // REQUIRED  
    creditAccount(); // REQUIRED  
}
```

🧠 Behavior:

- One single transaction is created and **shared** by all.
- If **any method fails**, the **entire transaction rolls back**.

✅ Use case: This is safest for business logic that should fully succeed or fail together.

✅ **2. REQUIRES_NEW**

Child suspends parent's transaction and starts a new one.

@Transactional

```
public void transferMoney() {  
    debitAccount(); // REQUIRED  
    creditAccount(); // REQUIRES_NEW  
}
```

🧠 Behavior:

- transferMoney() starts Transaction A
- creditAccount() **suspends Transaction A**, starts Transaction B
- If creditAccount() fails, only **Transaction B rolls back**, Transaction A can still commit
- But if debitAccount() fails, **Transaction A rolls back** (but B may have already committed!)

⚠️ Danger: This may lead to **inconsistent data** unless handled carefully.

✅ Use case: Use when a **sub-task must succeed independently**, like logging, audit, or sending notification.

✅ 3. NESTED

Child runs within parent, but rollback can be partial (uses savepoints).

@Transactional

```
public void transferMoney() {  
    debitAccount(); // REQUIRED  
    creditAccount(); // NESTED  
}
```

🧠 Behavior:

- All part of the same transaction.

- But if `creditAccount()` fails, **only that part can rollback** (if handled properly with try-catch).
- Parent continues, unless you throw an exception yourself.

✅ Use case: Useful when you want to isolate failures inside the same transaction without canceling everything.

⚠️ Works only with **JDBC and DataSourceTransactionManager**.

✅ 4. SUPPORTS

Child runs in a transaction if parent has one, else runs without.

@Transactional

```
public void transferMoney() {  
    debitAccount(); // SUPPORTS  
    creditAccount(); // SUPPORTS  
}
```

🧠 Behavior:

- If `transferMoney()` is transactional, children join.
- If it is not, children run without any transaction.

✅ Use case: Safe for **read-only** or optional transaction cases.

✅ 5. MANDATORY

Child must be called inside a transaction.

@Transactional

```
public void transferMoney() {  
    validateAccount(); // MANDATORY  
}
```

🧠 Behavior:

- If parent method does **not** have a transaction, Spring throws an exception.

✅ Use case: Force a method to **never run outside of a transaction** (e.g., updating sensitive data).

✅ 6. NEVER

Fails if there's an existing transaction.

@Transactional

```
public void transferMoney() {  
    notifyExternalAPI(); // NEVER  
}
```

🧠 Behavior:

- If notifyExternalAPI() is called **inside a transaction**, an exception is thrown.

✅ Use case: Use for calling **external systems** or **non-transactional operations**.

✅ 7. NOT_SUPPORTED

Suspends the current transaction and runs non-transactionally.

@Transactional

```
public void transferMoney() {  
    sendSMS(); // NOT_SUPPORTED  
}
```

🧠 Behavior:

- Suspends the active transaction to **isolate the call from rollback**.

✅ Use case: Used for external tasks like **logging, file I/O, or messaging**.

🧪 **Summary with Real-World Mapping**

Method	Propagation	What Happens	Use It For
debitAccount()	REQUIRED	Joins parent. Whole tx fails if any part fails	Normal business logic (safe default)
creditAccount()	REQUIRES_NEW	Runs in new tx. Can commit even if parent fails	Independent task (e.g., audit log)
validateAccount()	MANDATORY	Fails if parent has no tx	Critical checks requiring tx
notifyUser()	NEVER	Fails if parent has tx	External API calls
logAction()	NOT_SUPPORTED	Runs without tx, suspends parent	Logging, email, analytics
fetchHistory()	SUPPORTS	Uses tx if parent has it, else none	Read-only operations
creditWithTry()	NESTED	Savepoint rollback possible without killing parent	Partial rollback logic

Summary

- JPA uses EntityManager to manage transactions.
- Real transaction boundaries are controlled by:
 - The app server (JTA)
 - Or by frameworks like **Spring**
- @Transactional provides a **declarative way** to manage transaction scope.
- Internal flow involves:
 - Begin → Persist (in memory) → Flush → Commit or Rollback

- Rollbacks are automatic on **unchecked exceptions** (unless configured otherwise).

✓ Key JPA Concepts

1. Entity Lifecycle States

Understanding how entities behave in the persistence context is **foundational**.

State	Description
New	Object not associated with DB or persistence context
Managed	Tracked by EntityManager (will auto sync on commit)
Detached	Was managed earlier, now outside the persistence context
Removed	Marked for deletion

🔍 Example: Calling `em.detach(entity)` will make the entity unmanaged, so changes won't be saved.

2. Fetch Types – LAZY vs EAGER

`@OneToMany(fetch = FetchType.LAZY)` // default for `@OneToMany`

`private List<Order> orders;`

Type	Description
LAZY	Data is loaded only when accessed (preferred)
EAGER	Data is fetched immediately , even if not needed

⚠ **EAGER can cause performance issues and N+1 select problems.** Use LAZY by default and **fetch manually using JOIN FETCH or DTO projection.**

3. The N+1 Select Problem

When using EAGER or poorly written JPQL, you may hit multiple queries instead of one.

🔥 Fix using:

- JOIN FETCH in JPQL
 - Entity Graphs
 - DTO Projection with joins
-

4. JPQL vs Native Queries

- **JPQL**: Entity-based, database-agnostic.
- **Native Query**: SQL-based, more flexible, but DB-specific.

💡 Prefer JPQL unless performance or features demand native SQL.

5. DTO Projection

Instead of returning entire entities (which may trigger lazy loads), return **specific fields via DTO**:

```
@Query("SELECT new com.example.dto.StudentDTO(s.name, s.email) FROM Student s")
```

```
List<StudentDTO> getStudentSummaries();
```

✅ Faster, no lazy-loading issues, and cleaner APIs.

6. Entity Graphs

Used to **optimize fetching** without changing your query.

```
@EntityGraph(attributePaths = {"orders", "address"})
```

```
List<Customer> findAll();
```

🔗 Helps avoid N+1 problems **without writing custom JPQL**.

7. Optimistic vs Pessimistic Locking

✅ Optimistic Locking

@Version

private int version;

- Prevents lost updates using version field.
- Fails with OptimisticLockException if version doesn't match.

✅ Pessimistic Locking

@Lock(LockModeType.PESSIMISTIC_WRITE)

@Query("SELECT o FROM Order o WHERE o.id = :id")

Order lockOrder(@Param("id") Long id);

- Locks DB row immediately.
- Useful for concurrent updates on critical data.

8. Flush & Clear Behavior

- flush(): Syncs persistence context to DB
- clear(): Detaches all entities from context

💡 Needed when using native queries or batch operations to reduce memory usage.

9. Batch Inserts/Updates

Hibernate can batch queries for performance:

spring.jpa.properties.hibernate.jdbc.batch_size=50

✅ Use with caution: all entities must be in the same state.

10. Soft Deletes

Instead of deleting rows, mark them as inactive.

@Column(name = "is_deleted")

private boolean deleted;

Then use:

```
@Where(clause = "is_deleted = false")
```

✓ Keeps historical data and avoids foreign key issues.

11. Composite Keys (@EmbeddedId / @IdClass)

For tables with multiple primary keys:

```
@EmbeddedId
```

```
private OrderItemKey id;
```

Used in join tables or many-to-many relationships with extra fields.

12. Inheritance Strategies

```
@Inheritance(strategy = InheritanceType.JOINED)
```

JPA supports:

- SINGLE_TABLE: One big table (fast but sparse)
- JOINED: Normalized, multiple tables
- TABLE_PER_CLASS: Rarely used